A drift-aware and deployment-oriented evaluation framework for fish multi-object tracking in digital aquaculture

Abstract

This study presents a reproducible evaluation framework for fish multi-object tracking and counting in digital aquaculture, with an emphasis on drift-aware robustness analysis and deployment-oriented assessment. Rather than claiming across-the-board gains, we integrate run-configuration locking, data-ingestion checks, manifest hash tracing, controlled detector degradation (drop/jitter), quantile stratified difficulty analysis, count stability metrics, and runtime profiling in one reproducible pipeline. On a frozen val-half snapshot (8 sequences, 1000 frames each, seeds 0/1/2), Base reports HOTA 58.232 ± 0.304 and IDF1 64.042 ± 0.550, while ByteTrack reports HOTA 58.431 ± 0.291 and IDF1 64.011 ± 0.545. The +gating/+traj/+adaptive variants expose explicit trade-offs rather than monotonic gains. Controlled degradation and bucketed diagnostics show where failures accumulate (especially low-confidence and turning regimes), while runtime profiling separates lightweight options (Base, +gating) from high-cost options (+traj, +adaptive). We translate these observations into threshold-based guidance for CPU/edge deployment under frame-rate and memory constraints. The resulting framework links fair comparison, stress testing, failure analysis, and reproducible experimental reporting for smart aquaculture applications.

Keywords: digital aquaculture; multi-object tracking; fish counting; drift monitoring; reproducibility; runtime profiling; edge deployment.

1. Introduction

Digital aquaculture relies on reliable fish trajectories and counts for biomass estimation, feeding control, and welfare monitoring. In this context, a model that is slightly better on a single benchmark but hard to reproduce or deploy can be less useful than a model with transparent limits and well-characterized behavior. Therefore, this paper does not claim universal superiority of any single tracker. Instead, we present an evaluation framework that integrates fair comparison, robustness diagnostics, and deployment-oriented analysis under compute constraints.

Study scope and contribution:This manuscript contributes methodological advances for aquaculture vision evaluation, including a traceable benchmarking protocol, a drift-aware monitoring scheme, and a deployment-oriented analysis of accuracy–robustness–cost trade-offs. The goal is to improve reliable evaluation and deployment assessment for agricultural vision systems, rather than to propose a first-time application of an off-the-shelf tracker.

The contributions of this work are summarized as follows.

1. C1: A reproducible evaluation protocol for MOT and counting pipelines based on a locked experiment configuration, manifest hashing, and a release package. This component is intended to reduce inconsistencies in tables and figures caused by implicit path or setting changes.
2. C2: Drift-aware diagnostics and a stress-testing protocol based on controlled degradation, stratified risk buckets, and drift-monitoring signals. This component is intended to reveal failure concentration under turning, low-confidence, and degraded detection conditions that may not be visible from global mean metrics alone.
3. C3: A deployment-oriented comparison protocol based on joint tracking, counting, and resource evidence. In the current version, deployment interpretation is limited to Scope A measurements, and no end-to-end capacity claim is made without Scope B profiling.

2. Related wor

Tracking baselines and association design

Modern online MOT systems differ mainly in association design and handling of low-confidence detections. ByteTrack associates almost every detection and is a strong and efficient baseline (Zhang et al. 2022). OC-SORT emphasizes observation-centric updates under motion uncertainty (Cao et al. 2022). BoT-SORT integrates stronger association cues and camera-motion-aware design for improved robustness (Aharon et al. 2022). These methods form the strong-baseline set in our experiments.

Beyond these recent trackers, the broader MOT lineage (e.g., SORT/DeepSORT style association pipelines (Bewley et al. 2016; Wojke et al. 2017)) motivates two engineering concerns in practice: metric sensitivity to detection quality and speed-memory trade-offs under deployment constraints. Our framework addresses these concerns at protocol level rather than by proposing a new universal tracker.

Evaluation metrics and reproducible benchmarking

HOTA explicitly balances detection and association quality and is now a standard MOT metric (Luiten et al. 2021). TrackEval provides a unified implementation for metric computation and cross-method comparability (Luiten et al. 2020). We adopt this metric/evaluator stack and extend it with multi-seed aggregation, per-sequence reporting, stratified bucket analysis, and explicit reproducibility metadata.

Aquaculture vision, fish tracking, and counting

Aquaculture vision has evolved from foundational computer-vision task reviews toward full digital-aquaculture stacks that

integrate monitoring, management, and decision support (Zion 2012; Føre et al. 2018; Zhou et al. 2018; Li et al. 2020; Barbedo 2022; Cui et al. 2025). Along this trajectory, counting-oriented works have emphasized practical instrumentation and affordability, from embedded counters to vision-based hatchery/fingerling workflows and modern deep counting models (Hernandez-Ontiveros et al. 2018; Franca Albuquerque et al. 2019; Zhang et al. 2020). In parallel, tracking and behavior studies have shifted toward deep models for abnormal-event detection and monitoring robustness in dense farming scenes (Mei et al. 2022; Wang et al. 2022; Zhao et al. 2025).

However, most studies still report isolated task metrics (only counting, only tracking, or only behavior) and rarely connect these outcomes to reproducibility controls, stress-test protocols, and deployment constraints in one auditable pipeline. Our contribution lies in a drift-aware, stress-tested, and deployment-oriented evaluation framework for aquaculture MOT, rather than in proposing a new tracking architecture.

3. Materials and methods

3.1 Terminology and scope

Input drift denotes a distribution change between a reference window and an online window for input-side signals (e.g., detection confidence, box geometry, image quality proxies). Output drift denotes a change in output-side quality indicators (e.g., tracking metrics, counting errors, ID consistency proxies) under a fixed evaluation protocol. This work addresses drift-aware monitoring and response, not formal drift elimination guarantees.

3.2 Audit-ready reproducibility core

The framework starts from a single run configuration (run_config.json) that locks split, frame budget, seeds, degradation parameters, gating threshold, and data root. Drift risk is monitored by explicit MOT-root policy checks and data-ingestion gates, rather than assumed to be impossible. Outputs are routed into fixed subdirectories (tables, runtime, stratified, paper assets, release), while manifest hashing records provenance of both inputs and outputs.

3.2.1 Drift definition and closed-loop handling

Let $X_t^{(W)} = \{x_{t-W+1}, ..., x_t\}$ be the recent input window of size W and X_{ref} be a reference window (calibration period). Let $q(\cdot)$ denote output-quality statistics computed on a window. In this manuscript, W denotes the fixed monitoring window length configured before evaluation, and K denotes the persistence window count required to trigger an alert. The reference window X_{ref} is constructed from a calibration interval of the same stream under nominal operating conditions and then frozen for online monitoring. The exact calibration interval length and refresh policy are fixed in the released configuration and should be reported in the final archival version.

Input-layer indicator.

For a scalar feature z (e.g., detection confidence or gating score), we monitor the empirical CDF distance:

$$D_{\text{in}}(t) = \max_u \left| F_{z,t}^{(W)}(u) - F_{z,\text{ref}}(u) \right|$$

Output-layer indicator.

For output metric vector $q_t = [IDSW_t, CountMAE_t, F1_t]^T$, we monitor normalized deviation:

$$D_{\text{out}}(t) = \left\| \frac{q_t - \mu_{q,\text{ref}}}{\sigma_{q,\text{ref}} + \epsilon} \right\|_1$$

Here $\mu_{q,ref}$ and $\sigma_{q,ref}$ are reference statistics; ε avoids division by zero.

Trigger and calibration.

Drift alert is raised when $D_{in}(t) > \tau_{in}$ or $D_{out}(t) > \tau_{out}$ for K consecutive windows, where $K \in \mathbb{N}^+$ is the persistence hyper-parameter (number of consecutive windows required before alerting). Thresholds (τ_{in}, τ_{out}) are calibrated from reference quantiles (e.g., 95th or 99th percentile) and then stress-tested for false-alarm and missed-detection trade-offs. The reference-window length, refresh policy, and selected quantiles and K values should be documented in the final released configuration for full reproducibility.

Mitigation policy.

Upon alert, the system applies a conservative fallback sequence: enable/strengthen gating, adjust threshold using score quantiles, switch to a safer tracker profile, and log a re-calibration recommendation.

Algorithm 1. Drift-aware closed-loop monitoring and fallback

Input: windows $X_t^{(W)}$; output stats q_t; thresholds (τ_{in}, τ_{out}); persistence K; reference X_{ref}.

State: alert counter c <- 0; current profile p <- default.

For each window t: compute $D_{in}(t)$ and $D_{out}(t)$. If $D_{in}(t) > \tau_{in}$ or $D_{out}(t) > \tau_{out}$ then c <- c+1 else c <- 0.

If c >= K: activate conservative mode (e.g., tighter gating / safer tracker profile); log context {timestamp, $D_{in}(t)$, $D_{out}(t)$, c, profile, decision}.

The final fallback profile catalog and log schema should be documented in the released implementation, including field names and units.

3.2.2 Controlled degradation protocol

We evaluate robustness under controlled detection degradation using drop rate and bounding-box jitter. This protocol quantifies how rapidly tracking identity and matching quality degrade as sensor/detector quality worsens, which is central for aquaculture edge deployments where lighting/turbidity conditions vary.

3.2.3 Environment-aware stress tests (underwater/aquaculture)

Beyond drop/jitter, we define parameterized degradations aligned with aquaculture conditions:

1. Motion blur: convolve each frame I with a linear motion kernel k_{ℓ,θ}, where ℓ is blur length (pixels) and θ is motion direction.

$$I' = I * k_\{\ell, \theta\}$$

2. Low-light contrast shift: apply an affine intensity transform followed by clipping, where $\alpha \in (0,1)$ and $\beta < 0$ simulate illumination loss.

$$I' = clip(\alpha \cdot I + \beta)$$

3. Turbidity-like haze: apply a simple haze formation model with transmission $t \in (0,1)$ and ambient light A.

$$I' = I \cdot t + A \cdot (1 - t)$$

Each degradation is tested at three strengths (low/mid/high) with fixed random seed and identical sequence subsets across methods.

Environmental stress-test parameter settings

Degradation	Strength level	Parameter setting
Motion blur	low/mid/high	(ℓ, θ)
Low-light contrast shift	low/mid/high	(α, β)
Turbidity-like haze	low/mid/high	(t, A)

3.2.4 Stratified difficulty diagnostics

To expose failure mechanisms beyond global means, we run quantile-based stratified evaluation over occlusion, density, turning, and low-confidence scores. Each bucket reports F1, IDSW, and counting errors, enabling direct identification of difficulty regimes that trigger association instability.

3.2.5 Count stability and runtime profiling

We augment MOT metrics with count stability indicators (CountMAE/CountRMSE/CountVar/CountDrift) and runtime profiles under two scopes: A) tracking-only and B) end-to-end (decode + detect + track + output). This makes it possible to build threshold-based deployment rules rather than relying on one-score ranking.

3.3 Experimental setup

3.3.1 Frozen configuration and data

Configuration is locked in the frozen release package: split=val-half, MOT root=data/mft25_mot_full, max frames=1000, seeds=[0, 1, 2], drop rate=0.2, jitter=0.02, and gating threshold=2000.0. The evaluated val-half bundle contains eight sequences and 8000 evaluated frames in total.

3.3.2 Methods and fairness protocol

We evaluate Base, +gating, +traj, +adaptive, and strong baselines ByteTrack/OC-SORT/BoT-SORT. Fairness rule: all trackers consume the same detection source and are evaluated with the same split/frame budget and TrackEval pipeline. Reporting uses multi-seed mean±std when available.

3.3.3 Runtime measurement protocol

Scope A (tracking-only): measured on pre-loaded detections, including tracker state update, association, and output state generation, while excluding video decode, detector forward pass, and output serialization to disk. Scope B (end-to-end): includes decode + detection + tracking + output serialization under the same sequence list.

For each scope, the timing protocol is: (i) run N_warm warm-up iterations excluded from statistics, (ii) run N_rep timed repetitions with fixed thread settings and identical input order, and (iii) report central tendency and dispersion as mean±std across repetitions; when required, additionally report a confidence interval (CI) with the estimator and confidence level explicitly stated. The values of N_warm, N_rep, the CI type (if used), and the random-seed policy should be documented in the released runtime configuration.CPU reporting uses two fields: cpu_mean_1core_percent (as returned by the process monitor) and cpu_mean_norm_percent normalized by logical CPU count. Memory is reported as peak RSS in MB.

Runtime CSV fields used in the profiling scripts.

Field name	Definition
scope	{tracking-only, end-to-end}
elapsed_sec	Wall-clock seconds per run
fps	Processed frames / elapsed_sec
mem_peak_mb	Peak resident memory in MB
cpu_mean_1core_percent	Mean process CPU percentage (monitor-native)
cpu_mean_norm_percent	cpu_mean_1core_percent / logical CPU count
warmup_iters	Number of warm-up iterations before timing
repeat_runs	Number of repeated timed runs

3.3.4 Research questions

RQ1: How do core variants and strong baselines compare on global metrics?

RQ2: What robustness trend appears under controlled degradation?

RQ3: Which difficulty buckets dominate ID switches and counting errors?

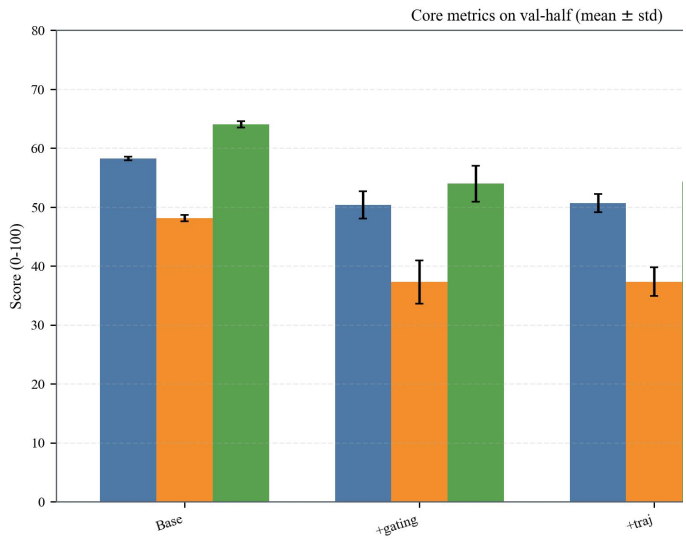
RQ4: Which methods satisfy deployment budgets on FPS/memory/CPU?

4. Results

4.1 Core method family (Base to +adaptive)

Table and Fig. 1 answer RQ1 for the internal method family. Base achieves HOTA 58.232 ± 0.304 and IDF1 64.042 ± 0.550. In this frozen run, +gating/+traj/+adaptive are below Base on HOTA/IDF1 and above Base on IDSW, so they are interpreted as trade-off variants rather than monotonic improvements..

Method	HOTA	DetA	AssA	IDF1	IDSW
Base	58.232 ± 0.304	73.150 ± 0.037	48.121 ± 0.577	64.042 ± 0.550	79.375 ± 1.203
+gating	50.345 ± 2.317	72.504 ± 0.033	37.281 ± 3.652	53.967 ± 3.051	130.458 ± 0.425
+traj	50.695 ± 1.557	72.448 ± 0.135	37.332 ± 2.443	54.277 ± 1.996	117.792 ± 0.680
+adaptive	51.071 ± 1.435	72.538 ± 0.226	37.771 ± 2.167	54.667 ± 1.779	113.792 ± 0.257



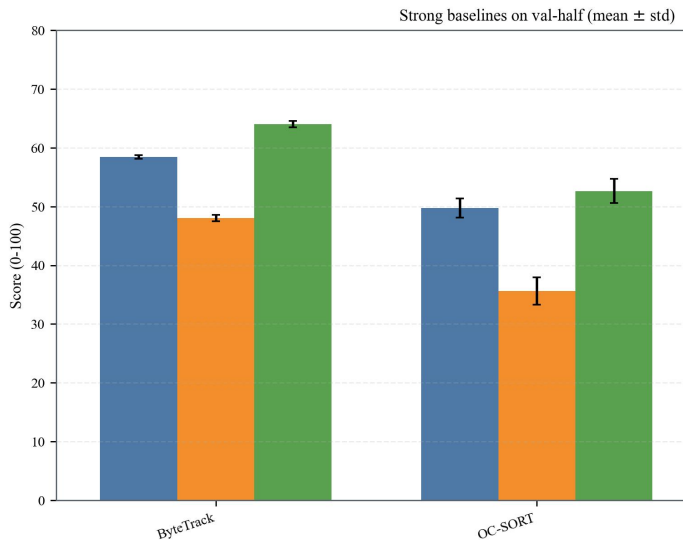
Core method family comparison (mean±std).

4.2 Strong baselines and combined comparison

Table and Fig. 2 answer RQ1 for strong baselines only. ByteTrack is the strongest among the three baselines on HOTA (58.431 ± 0.291) and IDF1 (64.011 ± 0.545) in this run.

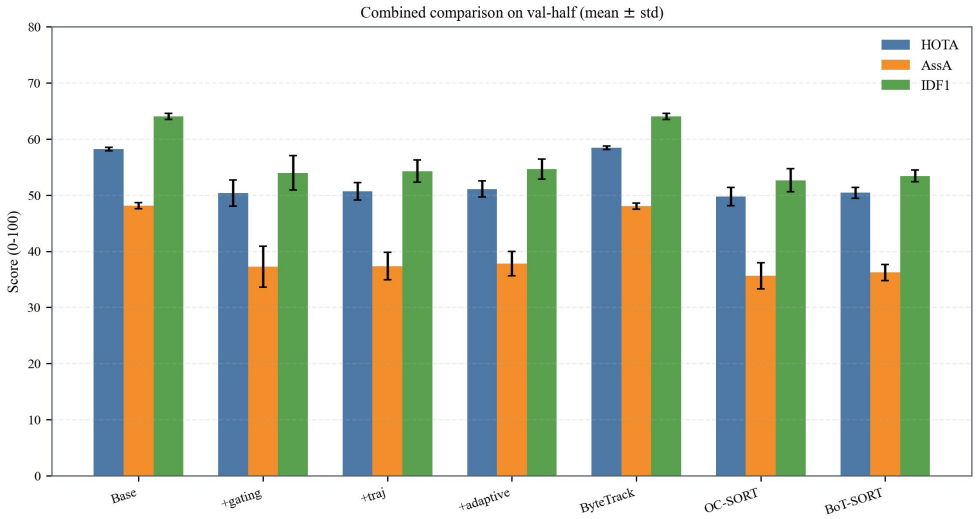
To show fair side-by-side context, Table 3 and Fig. 3 combine all methods under one evaluation pipeline. ByteTrack exceeds Base by +0.199 HOTA, while Base has lower IDSW by 9.000 compared with ByteTrack. These results support a trade-off interpretation rather than a single-winner claim.

Method	HOTA	DetA	AssA	IDF1	IDSW
ByteTrack	58.431 ± 0.291	73.858 ± 0.015	48.049 ± 0.569	64.011 ± 0.545	88.375 ± 0.872
OC-SORT	49.732 ± 1.619	73.985 ± 0.135	35.605 ± 2.352	52.631 ± 2.057	145.875 ± 1.643
BoT-SORT	50.414 ± 0.981	73.426 ± 0.087	36.214 ± 1.425	53.412 ± 1.047	121.000 ± 4.056



Strong baseline comparison (ByteTrack/OC-SORT/BoT-SORT).

Method	HOTA	DetA	AssA	IDF1	IDSW
Base	58.232 ± 0.304	73.150 ± 0.037	48.121 ± 0.577	64.042 ± 0.550	79.375 ± 1.203
+gating	50.345 ± 2.317	72.504 ± 0.033	37.281 ± 3.652	53.967 ± 3.051	130.458 ± 0.425
+traj	50.695 ± 1.557	72.448 ± 0.135	37.332 ± 2.443	54.277 ± 1.996	117.792 ± 0.680
+adaptive	51.071 ± 1.435	72.538 ± 0.226	37.771 ± 2.167	54.667 ± 1.779	113.792 ± 0.257
ByteTrack	58.431 ± 0.291	73.858 ± 0.015	48.049 ± 0.569	64.011 ± 0.545	88.375 ± 0.872
OC-SORT	49.732 ± 1.619	73.985 ± 0.135	35.605 ± 2.352	52.631 ± 2.057	145.875 ± 1.643
BoT-SORT	50.414 ± 0.981	73.426 ± 0.087	36.214 ± 1.425	53.412 ± 1.047	121.000 ± 4.056

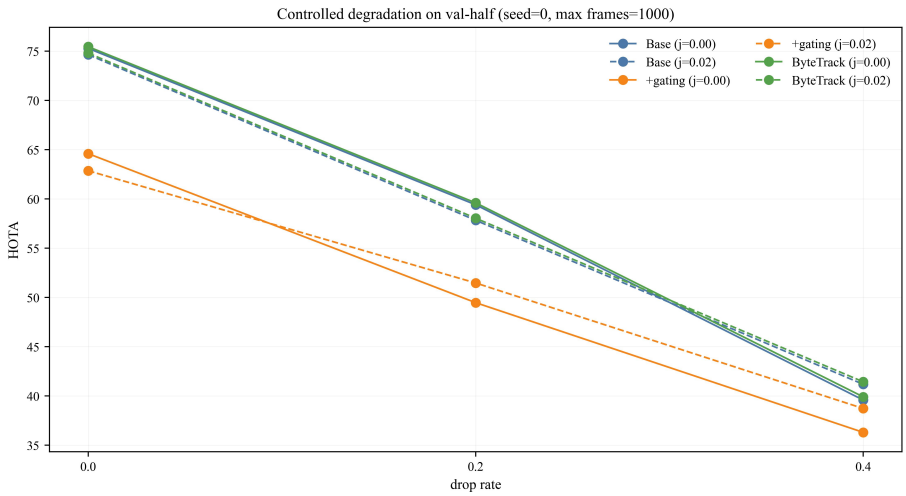


Combined comparison of proposed variants and strong baselines.

4.3 Robustness under controlled degradation

Using the frozen degradation table, we compute drop-induced degradation from drop rate 0.0 to 0.4 at jitter=0.00 and 0.02 (Table). For jitter=0.00, HOTA/IDF1 are: Base (35.707/27.959), +gating (28.299/19.257), ByteTrack (35.557/27.857). For jitter=0.02: Base (33.474/25.141), +gating (24.139/14.888), ByteTrack (33.382/25.082). At the worst point (drop=0.4, jitter=0.02), HOTA ranking is ByteTrack 41.435 > Base 41.182 > +gating 38.727; the top-vs-second gap is 0.253. IDF1 ties between ByteTrack and Base at 49.770, both above +gating (45.989). For IDSW (lower is better), Base (99.625) < ByteTrack (112.750) < +gating (127.875).

Jitter	Base HOTA	Base IDF1	+gating HOTA	+gating IDF1	ByteTrack HOTA	ByteTrack IDF1
	35.707	27.959	28.299	19.257	35.557	27.857
0.02	33.474	25.141	24.139	14.888	33.382	25.082

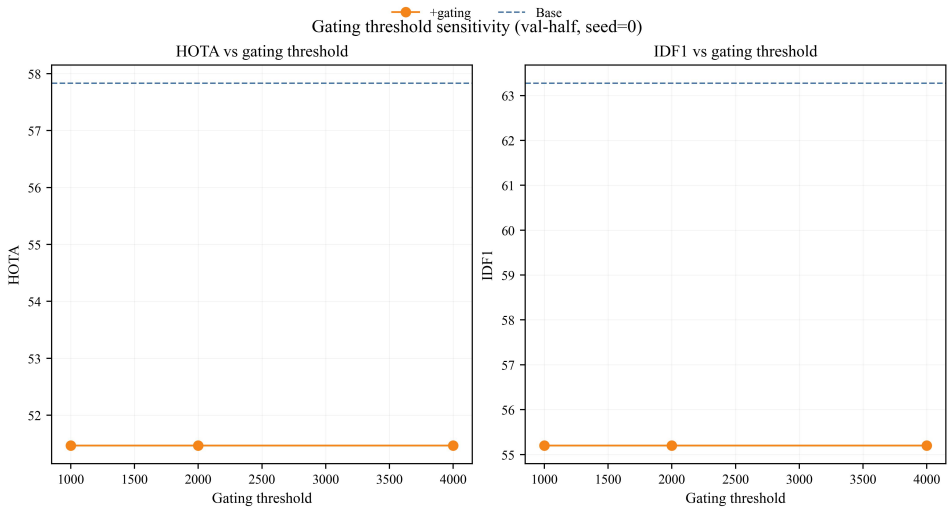


Controlled degradation curves (Base, +gating, ByteTrack).

4.4 Gating threshold sensitivity (deployment robustness check)

Using the frozen sensitivity table (seed=0), Base reports HOTA/IDF1/IDSW = 57.831/63.274/79.000, while +gating at thresholds 1000, 2000, and 4000 is identical at 51.469/55.202/130.875. Relative to Base, this corresponds to -6.362 HOTA, -8.072 IDF1, and +51.875 IDSW for all three threshold points. The overlap of 1000/2000/4000 defines a candidate plateau interval under the current split/seed. We therefore use 2000 as the default operating point within this interval, while the activation analysis in Section 5.5 is intended to assess whether this pattern reflects true threshold insensitivity or limited score support in the sampled data.

Method	Gating threshold	HOTA	DetA	AssA	IDF1	IDSW
Base	N/A	57.831	73.140	47.320	63.274	79.000
+gating	1000	51.469	72.524	39.015	55.202	130.875
+gating	2000	51.469	72.524	39.015	55.202	130.875
+gating	4000	51.469	72.524	39.015	55.202	130.875



Sensitivity of +gating to threshold values (1000/2000/4000) at seed=0.

4.5 Gating activation analysis

To further interpret the candidate plateau interval, we analyze activation-level diagnostics of the gating mechanism. In the current implementation, the association cost is defined as:

$$C_{ij} = \alpha\left(1 - \mathrm{IoU}_{ij}\right) + \beta\, d^2_{\mathrm{maha},ij} + \gamma\, c^{\mathrm{traj}}_{ij}$$

Candidate pairs are rejected when the gating score exceeds the threshold \tau_g. The gating score is defined as:

$$s^{\mathrm{gate}}_{ij} = d^2_{\mathrm{maha},ij}$$

where both the gating score and threshold τ_g are unitless squared-Mahalanobis quantities (computed in the association step).

To characterize the behavior of the gating mechanism under different thresholds, we consider the following activation-level statistics:

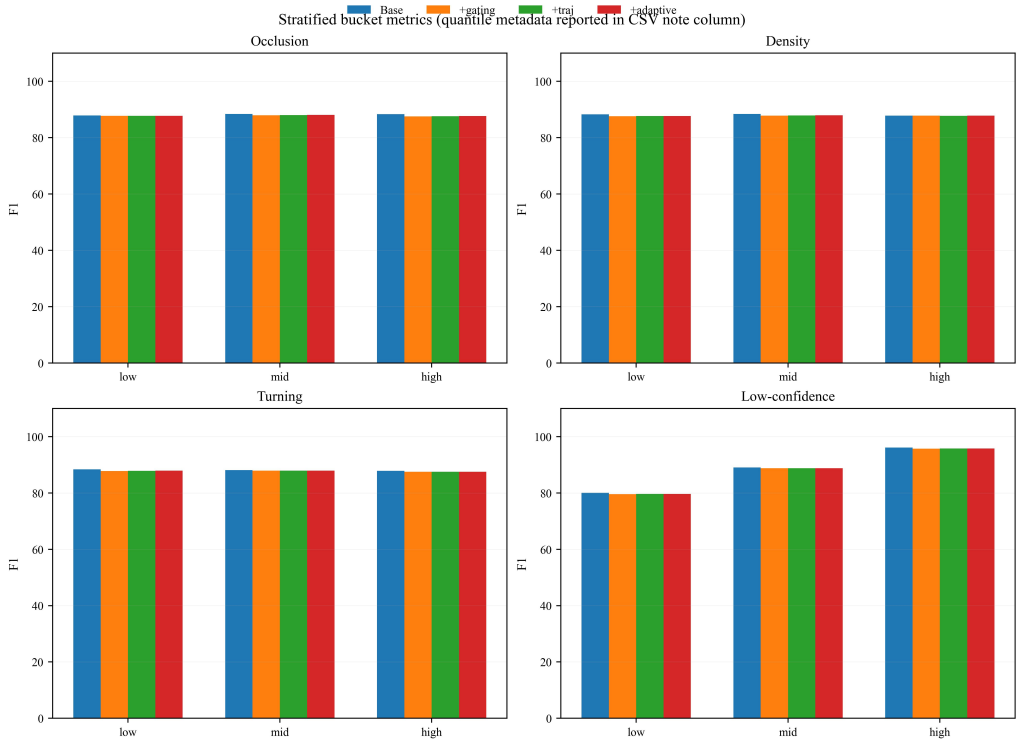
These statistics include the trigger rate at both the frame and sequence levels, the distribution of trigger-event lengths measured as consecutive activated frames, and the distribution of gating scores summarized by cumulative distribution functions or histograms with threshold markers.

To avoid threshold choices based only on manually specified constants, the sweep range is defined using score quantiles, for example from the 5th to the 95th percentile.

4.6 Stratified difficulty and failure modes

Figure 6 answers RQ3 by showing bucket-wise metrics. For Base, high-risk buckets include turning (mid) (IDSW=627, CountMAE=4.795), low-confidence (mid) (IDSW=612, CountMAE=4.002), and occlusion (low) (IDSW=569, CountMAE=4.655).

Relative to Base, +gating increases IDSW in the low-confidence (mid) bucket from 612 to 984 (+372) and raises CountMAE in occlusion (high) from 1.741 to 1.848 (+0.107). This indicates that low-confidence and turning regimes are key failure modes for identity consistency.

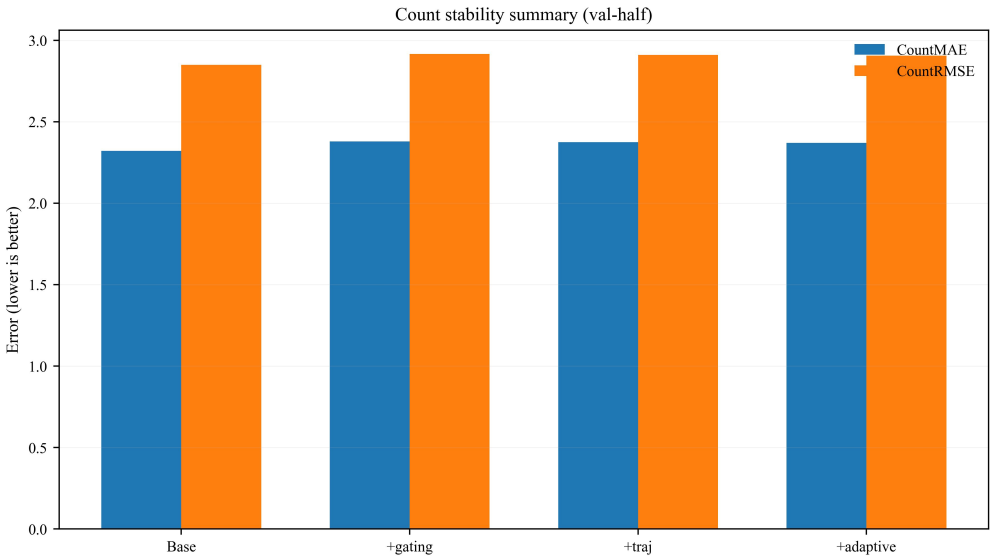


Stratified metrics over occlusion, density, turning, and low-confidence buckets; quantile and fallback notes are reported in stratified_metrics_val.csv.

4.7 Counting stability

Table 6 and Fig. 7 answer RQ3 from a counting perspective. Mean CountMAE is 2.321 (Base), 2.379 (+gating), 2.375 (+traj), and 2.371 (+adaptive). Mean CountRMSE follows the same order (2.850, 2.916, 2.911, 2.906).

Method	CountMAE	CountRMSE	CountVar	CountDrift
Base	2.321	2.850	3.746	2.075
+gating	2.379	2.916	3.825	2.075
+traj	2.375	2.911	3.817	2.083
+adaptive	2.371	2.906	3.808	2.100



Count stability comparison.

4.8 Runtime and resource profile

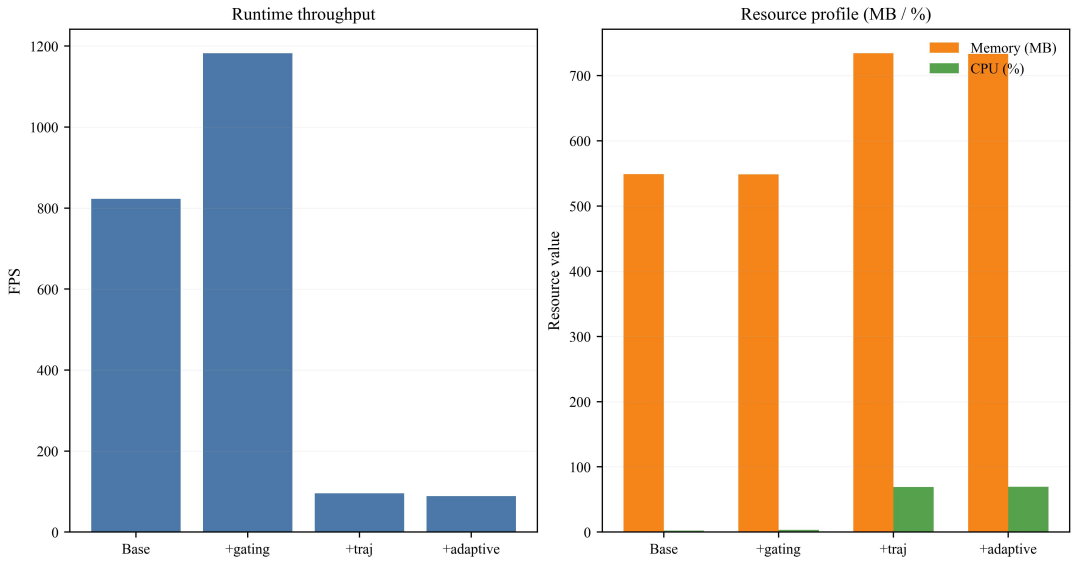
Figure 8 answers RQ4 for Scope A (tracking-only). Base runs at 822.812 FPS with 548.992 MB peak memory and 2.157% normalized CPU. +gating is 1182.212 FPS, 548.395 MB, 3.355%. +traj and +adaptive are much heavier: 95.740/89.024 FPS and 734.211/733.199 MB, with CPU around 69.259%/69.488%.

We report Scope A and Scope B separately because they answer different deployment questions. Scope A isolates tracker-internal efficiency under controlled inputs, while Scope B captures user-facing performance when decode and detection overhead are included in the critical path. This separation avoids over-interpreting tracker-only throughput as end-to-end system capacity.

Scope B (end-to-end) results are not included in the current version. Therefore, this manuscript does not make end-to-end deployment capacity claims.sion.

Scope B reporting items to be included in future end-to-end profiling.

Field (csv)	Reporting requirement (no values filled in this manuscript)
fps_e2e	End-to-end throughput including decode+detect+track+write.
mem_peak_mb_e2e	Peak resident memory in end-to-end runs.
cpu_norm_e2e	Normalized CPU usage in end-to-end runs.
detector_time	Detector-stage elapsed time.
decode_time	Decode-stage elapsed time.
tracking_time	Tracker-stage elapsed time in end-to-end runs.
write_time	Output serialization/write elapsed time.
detector_name	Detector model identifier and version.
input_resolution	Input resolution (W×H) used in end-to-end runs.



Runtime profiling (FPS, memory, normalized CPU).

5. Discussion

The results suggest that deployment decisions should be based on operating intervals and resource constraints rather than on a single aggregate score. Gating-threshold selection is therefore interpreted as a candidate operating interval rather than a single tuned constant: under the current split/seed, 1000/2000/4000 show identical +gating metrics, and Section 5.5 specifies the activation diagnostics required to determine whether this observation reflects true insensitivity or limited score support.

- If the deployment target is at least 30 FPS and the memory budget is up to 600 MB, Base and +gating satisfy tracker-internal throughput and memory constraints under Scope A. However, end-to-end throughput must be verified using Scope B measurements before any deployment-capacity claim is made.
- If heavier computation is acceptable, such as in server-side analysis, +traj and +adaptive can be considered, but their speed, memory, and CPU costs should be justified by task-specific value.
- Low-confidence and turning buckets show marked IDSW growth, suggesting that online confidence alarms or conservative association policies may be useful under these conditions.

Together, these results connect global benchmark metrics, failure-bucket diagnostics, and resource constraints to deployment-oriented interpretation.

5.1 Extension protocol for cross-domain validation

To improve external validity in future work, we outline a staged protocol with three progressively higher-cost phases:

1. Stage 1 (cross-day, same pond/device). Calibrate on Day-A and evaluate on Day-B. Transfer tracker hyper-parameters directly; re-estimate only drift-trigger thresholds when needed.

2. Stage 2 (cross-pond, same device). Calibrate on Pond-1 and evaluate on Pond-2. Keep association design fixed; permit short-window threshold re-calibration.
3. Stage 3 (cross-device or public dataset transfer). Use farm-calibrated settings as zero-shot initialization, then perform limited re-calibration of confidence and drift-trigger thresholds.

For each plan, report overall metrics (HOTA, IDF1, IDSW, CountMAE), drift indicators (D_in, D_out), stratified bucket risk shift, and gating trigger-rate shift.

Suggested reporting items for future cross-domain validation

- Fig G1: in-domain vs cross-domain radar (HOTA/IDF1/IDSW/CountMAE).
- Fig G2: drift indicator trajectories (D_in, D_out) across domains.
- Table G1: transferable vs re-calibrated parameters per plan.

Failure interpretation protocol.

If cross-domain performance degrades, we report whether degradation is dominated by (i) input drift (feature distribution shift), (ii) output drift (IDSW/CountMAE escalation), or (iii) both; and map each case to mitigation steps (threshold re-calibration, conservative mode, or detector refresh).

5.2 Nomenclature / notation

Core symbols used in drift-aware monitoring and counting diagnostics.

Symbol	Definition
D_in	Input-drift indicator (windowed CDF distance to reference).
D_out	Output-drift indicator (normalized quality deviation).
s_gate	Gating score, defined as squared Mahalanobis distance in association.
τ_g	Gating threshold applied to s_gate.
cpu_mean_norm_percent	Normalized CPU percentage, defined as cpu_mean_1core_percent / N_logical_cpu.
CountMAE	Mean absolute counting error per evaluation unit.
CountRMSE	Root mean squared counting error per evaluation unit.
CountVar	Variance of counting error across evaluation units.
CountDrift	temporal drift statistic of counting error over a window of size W_d. The value of W_d and the reported unit should be fixed according to the frozen script output fields.

5.3 Limitations

This study has several limitations. First, the evaluation scope is limited to the current val-half split and detector condition; broader domain shifts (different farms, sensors, water conditions) are not fully covered. Second, our variants do not show universal global metric gains in this frozen run, so conclusions are intentionally framed as trade-offs. Third, runtime profiling is collected on the current host and does not yet include dedicated edge hardware benchmarks. Fourth, although reproducibility metadata is strong, data sharing may still be constrained by upstream licensing.

6. Conclusions and future work

We presented a drift-aware and deployment-oriented evaluation framework for fish MOT and counting that emphasizes robustness–cost analysis, failure transparency, and deployment guidance. The framework combines fair baseline comparison, controlled degradation, stratified diagnostics, count stability analysis, and runtime profiling within one reproducible pipeline.

Future work for the smart-aquaculture PhD direction includes: (i) multi-sensor fusion with environmental signals (e.g., dissolved oxygen and water temperature), (ii) anomaly behavior recognition with trajectory-context coupling, (iii) cross-pond/domain generalization, and (iv) online updating/calibration strategies under long-term deployment drift.

7. Data availability and reproducibility statement

Code availability. Source code and experiment scripts are available to editors and reviewers through an anonymized release package. A public repository URL and archival DOI will be provided after acceptance.

Data availability. Raw farm videos may be subject to licensing and operational restrictions. When redistribution is limited, we provide derived benchmark artifacts, including predictions, per-sequence metrics, aggregated tables, and figure assets, required to reproduce the reported results. The final data-access statement and repository identifier will be provided in accordance with the applicable license constraints.

Reproducibility package. The package includes a locked run configuration, provenance metadata, cryptographic hash records, and a one-click reproduction script. This package is intended to support independent regeneration of the reported results without manual path reconstruction.

Appendix A. Reviewer release package

For editorial and reviewer access, the reproducibility package is shared through anonymized links during double-blind review.

- Reviewer access is provided through the submission system during double-blind review. Public archival identifiers, such as a DOI, will be added after

acceptance in accordance with data and code policies.

The release package contains run configuration, manifest and hash records, aggregated tables, paper assets, and one-click reproduction scripts.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Author contributions (CRediT)

Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization: author team. Supervision, Project administration, Resources: author team. All authors read and approved the final manuscript.

Competing interests

The authors declare no competing interests.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI Codex/GPT-based tools to improve language clarity, section organization, and editorial consistency. All scientific content, experiment execution, numerical verification, and final editing decisions were manually performed and validated by the authors, who take full responsibility for the manuscript. No generative AI tools were used to create or modify figures, graphical abstracts, or other artwork in this manuscript.

References

Aharon, Nir, Roy Orfaig, and Ben-Zion Bobrovsky. 2022. BoT-SORT: Robust Associations Multi-Pedestrian Tracking. https://arxiv.org/abs/2206.14651.

Barbedo, Jayme Garcia Arnal. 2022. A Review on Computer Vision-Based Recognition and Monitoring of Fish. Fishes 7 (6): 335. https://doi.org/10.3390/fishes7060335.

Bewley, Alex, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. 2016. Simple Online and Realtime Tracking. IEEE International Conference on Image Processing Workshops.

Cao, Jinkun, Xinshuo Weng, Rawal Khirodkar, Jiangmiao Pang, and Kris M. Kitani. 2022. OC-SORT: Observation-Centric SORT for Robust Multi-Object Tracking. https://arxiv.org/abs/2203.14360.

Cui, Tianfei, Jian Wang, Min Shen, et al. 2025. A Comprehensive Survey of Artificial Intelligence in Fish Farming. Reviews in Aquaculture 17 (1): e13001. https://doi.org/10.1111/raq.13001.

Føre, Martin, Kevin Frank, Tomas Norton, et al. 2018. Precision Fish Farming: A New Framework to Improve Production in Aquaculture. Biosystems Engineering 173: 176–93. https://doi.org/10.1016/j.biosystemseng.2017.10.014.

Franca Albuquerque, Caroline, Rafael Minho, Taiana de Almeida Nunes, et al. 2019. Automatic Live Fingerlings Counting Using Computer Vision. Computers and Electronics in Agriculture 167: 105015. https://doi.org/10.1016/j.compag.2019.105015.

Hernandez-Ontiveros, Juan Miguel, Everardo Inzunza-Gonzalez, Eduardo E. Garcia-Guerrero, et al. 2018. Development and Implementation of a Fish Counter by Using an Embedded System. Computers and Electronics in Agriculture 145: 53–62. https://doi.org/10.1016/j.compag.2017.12.023.

Li, Daoliang, Yinfeng Hao, and Yanqing Duan. 2020. Nonintrusive Methods for Biomass Estimation in Aquaculture with Emphasis on Fish: A Review. Reviews in Aquaculture 12 (3): 1390–411. https://doi.org/10.1111/raq.12388.

Luiten, Jonathon, Aljosa Osep, Patrick Dendorfer, Andreas Geiger, Laura Leal-Taixe, et al. 2020. TrackEval: A Unified Framework for Multi-Object Tracking Evaluation. https://arxiv.org/abs/2011.04605.

Luiten, Jonathon, Aljosa Osep, Patrick Dendorfer, Andreas Geiger, Laura Leal-Taixe, et al. 2021. Higher Order Tracking Accuracy: A New Metric for Multi-Object Tracking. International Journal of Computer Vision 129 (2): 548–78.

Mei, Jing, Jian Wang, Daoliang Li, et al. 2022. Recent Advances of Target Tracking Applications in Aquaculture with Emphasis on Fish. Computers and Electronics in Agriculture 201: 107335. https://doi.org/10.1016/j.compag.2022.107335.

Wang, He, Song Zhang, Shili Zhao, Qi Wang, Daoliang Li, and Ran Zhao. 2022. Real-Time Detection and Tracking of Fish Abnormal Behavior Based on Improved YOLOV5 and SiamRPN++. Computers and Electronics in Agriculture 192: 106512. https://doi.org/10.1016/j.compag.2021.106512.

Wojke, Nicolai, Alex Bewley, and Dietrich Paulus. 2017. Simple Online and Realtime Tracking with a Deep Association Metric. IEEE International Conference on Image Processing.

Zhang, Yifu, Peize Sun, Yi Jiang, et al. 2022. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. European Conference on Computer Vision.

Zhang, Zhaofeng, Yulong Zhang, Li Chen, Jian Tang, and Lucheng Cheng. 2020. Automatic Fish Population Counting by Machine Vision and a Hybrid Deep Neural

Network Model. Animals 10 (2): 364. https://doi.org/10.3390/ani10020364.

Zhao, Yaxuan, Hanxiang Qin, Ling Xu, Huihui Yu, and Yingyi Chen. 2025. A Review of Deep Learning-Based Stereo Vision Techniques for Phenotype Feature and Behavioral Analysis of Fish in Aquaculture. Artificial Intelligence Review 58. https://doi.org/10.1007/s10462-024-10960-7.

Zhou, Chao, Daming Xu, Kai Lin, Chuanheng Sun, and Xinting Yang. 2018. Intelligent Feeding Control Methods in Aquaculture with an Emphasis on Fish: A Review. Reviews in Aquaculture 10 (4): 975–93. https://doi.org/10.1111/raq.12218.

Zion, Boaz. 2012. The Use of Computer Vision Technologies in Aquaculture—a Review. Computers and Electronics in Agriculture 88: 125–32. https://doi.org/10.1016/j.compag.2012.07.010.